# Apps Script Web App Deployment Guide

From Google Sheet to deployed beginner-friendly web app.

**Big idea:** The web app works best when AI understands the whole workflow: what users should see, what data the app reads, what data it stores, who is allowed in, and how it should be deployed safely.

## Quick Reference: Tell AI These Things

| Prompt area | What to include |
|---|---|
| **Goal** | What should the tool help staff do faster, more clearly, or more consistently? |
| **Files needed** | Output two separate files: `Code.gs` and `index.html`. |
| **What users see** | Form, dashboard, search page, charts, buttons, confirmation screen, or document links. |
| **Data it reads** | Existing Sheet tabs, columns, or school workflow data the app should use. |
| **Data it stores** | Submissions, notes, status changes, settings, user roles, action logs, or error logs. |
| **Setup rule** | Any needed tabs and headers should be created automatically the first time the web app opens. |
| **Sharing rule** | Who should be able to open the web app, especially if student data is involved. |

## Beginner Workflow

| Step | What to do |
|---|---|
| **1. Start in Sheets** | Open the Sheet that will hold settings, data, logs, or submissions. |
| **2. Open Apps Script** | Click **Extensions > Apps Script**. |
| **3. Prompt AI** | Describe the workflow and ask for `Code.gs`, `index.html`, setup, testing, and deployment steps. |
| **4. Paste and test** | Paste each file, save, authorize if prompted, and test with sample data. |
| **5. Deploy** | Use **Deploy > New deployment > Web app**. |
| **6. Update safely** | After code changes, create a new version and update the deployment. |

# Build Request Before Code

What beginners should tell AI before pasting anything.

## Start From The Sheet

1. Open or create the Google Sheet that will support the web app.
2. Click **Extensions > Apps Script**.
3. Rename the Apps Script project so it matches the tool.
4. Tell AI to output two separate files: `Code.gs` and `index.html`.
5. Paste the server code into `Code.gs`.
6. Create or open the HTML file and paste the page code into `index.html`.

## What AI Needs To Know

Beginners do not need to explain code. They need to explain the school workflow clearly.

| Prompt area | What to explain |
|---|---|
| **Goal** | What should the web app help staff do faster, more clearly, or more consistently? |
| **Users** | Who will open it: office staff, counselors, admin, teachers, families, or students? |
| **Screen** | What should users see: form, dashboard, search page, charts, buttons, or documents? |
| **Read data** | What existing Sheet tabs should the app use, if any? |
| **Store data** | What should the app save: submissions, notes, status changes, logs, settings, or user roles? |
| **Outputs** | Should it send email, create Docs, make calendar events, update a dashboard, or return a link? |
| **Safety** | Who is allowed to view the tool, especially if it includes student data? |

## The One Setup Requirement To Include

If the tool saves data, tell AI that any required storage tabs must be created automatically the first time the web app opens. This prevents a beginner from missing a hidden setup step.

**Include this instruction:** Any Sheet tabs the web app needs for storage, settings, user roles, action logs, or error logs should be created automatically by the code the first time the web app opens. The code should also add the correct column headers.

# Student Data And Access

Simple safety choices before the web app is shared.

## Use An Admin Email Tab When Sharing Private Tools

If only you will use the web app, keep the deployment set to **Who has access: Only myself**. You do not need an `admin_email` tab for a personal testing tool.

If you plan to share the web app with other staff and it may show student names, attendance, behavior notes, intervention notes, family contact information, or staff-only records, access should be planned before the tool is deployed.

1. Create a Sheet tab named `admin_email`.
2. Put approved user emails in column A.
3. Tell AI that only those emails should be allowed into the web app.
4. If a user is not approved, the app should show an Access Denied page.
5. The app should log blocked attempts without saving sensitive student details.

**Important:** Email-based access works best inside a Google Workspace school domain where people sign in with school accounts. Test with an approved account and a blocked account before sharing the live link.

## Deployment Settings

In Apps Script, deploy through **Deploy > New deployment > Web app**. These two settings matter most.

| Setting | Beginner guidance |
|---|---|
| **Execute as: Me** | Usually best for beginner school tools. The app can read/write using your script permissions. Pair this with access checks when sensitive data is involved. |
| **Execute as: User accessing the web app** | Advanced. Each user may need their own permissions or authorization. Use this only when the tool truly needs to act as the person opening it. |
| **Who has access: Only myself** | Best for early testing or for a tool only you will use. |
| **Who has access: Anyone in your domain** | Often the best sharing option for internal school or district tools. Add role checks if student or staff data appears. |
| **Google account or anyone with link** | Avoid for student data unless the tool has been carefully designed, locked down, and tested. |

**FERPA-minded sharing habit:** If the tool contains student data, avoid public link access. Prefer domain-limited sharing plus a role check such as `admin_email`.

# Updating And Troubleshooting

What to do after code changes or something breaks.

## Versions And Updates

A deployed web app uses a saved version of the code. After changing `Code.gs` or `index.html`, the live link will not show those changes until the deployment is updated.

**This is an iterative process:** You can go back to AI to fix errors, add features, change the page design, improve buttons, adjust wording, or troubleshoot something that is not working. After you paste the updated code and save it, you must redeploy through **Deploy > Manage deployments** so the public web app link uses the newest version.

1. Ask AI for the fix, new feature, or change you want.
2. Paste the updated `Code.gs` or `index.html` code and save the project.
3. Test the tool from the Apps Script editor when possible.
4. Go to **Deploy > Manage deployments**.
5. Edit the active deployment, choose **New version**, and add a short note describing what changed.
6. Deploy and test the live web app link again.

Use the test URL while building. Share the live URL only after updating the deployment version.

## Ask AI To Add Helpful Logs

Logs should help diagnose problems without exposing private student details. Ask AI for:

- Friendly messages for users when something fails.
- Behind-the-scenes logs for important actions and errors.
- Sheet-based `Action Log` and `Error Log` tabs when the tool is important or shared.

**Good logging instruction:** Log timestamps, action names, record IDs, and short error messages. Do not log full student records, private notes, full emails, or sensitive family details.

## Where To Find Errors

| Place to check | What it helps with |
|---|---|
| **Execution log** | In Apps Script, run or test a function and open the Execution log to see recent messages. |
| **Executions tab** | On the left side of Apps Script, click Executions to see runs, failures, timeouts, and error messages. |
| **Browser console** | Use this when a button, form, page load, or dashboard interaction breaks inside the web app page. |
| **Windows / Chromebook** | In Chrome or Edge, press **F12** or **Ctrl + Shift + J**, then open the Console tab. |
| **Mac** | In Chrome or Edge, press **Command + Option + J**. In Safari, enable the Develop menu first, then open the JavaScript console. |

# Advanced Options And Checklist

Triggers, testing, and final sharing checks.

## Triggers

Triggers are advanced features that make Apps Script run automatically. They can be powerful, but they should be tested carefully because they can send emails, update Sheets, create files, or change records without someone clicking a button at that moment.

| Trigger type | School example |
|---|---|
| **Time-driven** | Run a daily attendance summary, weekly reminder check, or hourly dashboard refresh. |
| **Form submit** | Run after a Google Form submission, such as sending a notification or saving a follow-up task. |
| **Open/edit/change** | Respond when a Sheet is opened, edited, or structurally changed. |

When asking AI for a trigger, explain exactly what should happen, how often it should happen, what data it reads, what it changes, and how success or failure should be logged.

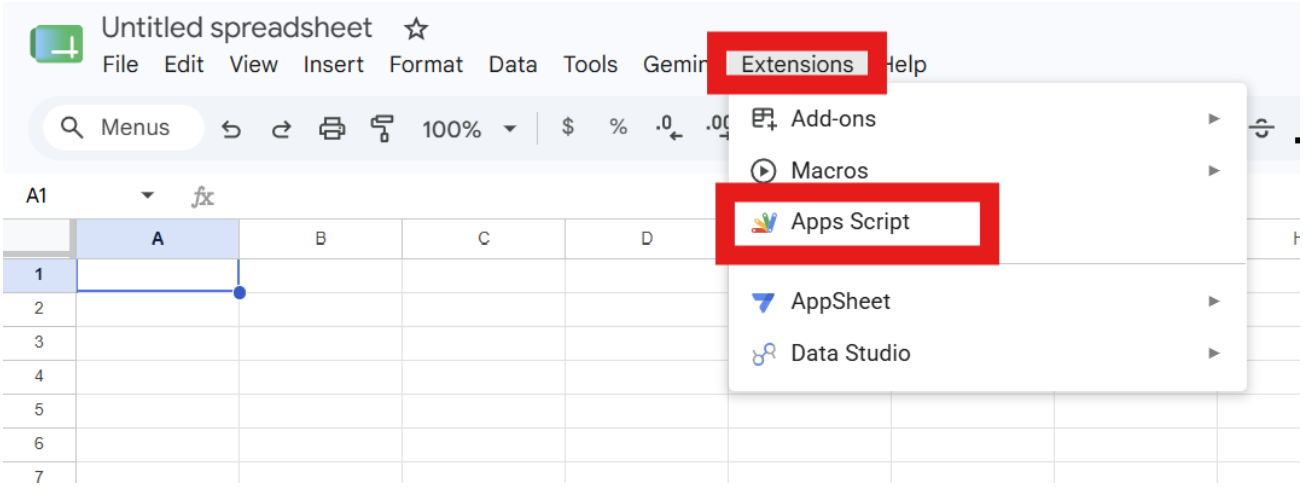
## Final Beginner Checklist

| Checkpoint | What to confirm |
|---|---|
| **Before pasting code** | AI gave separate `Code.gs` and `index.html` sections. AI explained where each one goes. |
| **Before testing** | The app creates any needed tabs and headers automatically. You are using sample data first. |
| **Before deploying** | You know whether the app runs as Me or as the user accessing the web app. |
| **Before sharing** | Access is limited appropriately for the data involved. Student-data tools are not public links. |
| **Before going live** | You tested buttons, forms, charts, emails, Docs, searches, access denied behavior, and logs. |
| **After edits** | You saved the updated code, created a new version in **Deploy > Manage deployments**, redeployed, and tested the live link again. |

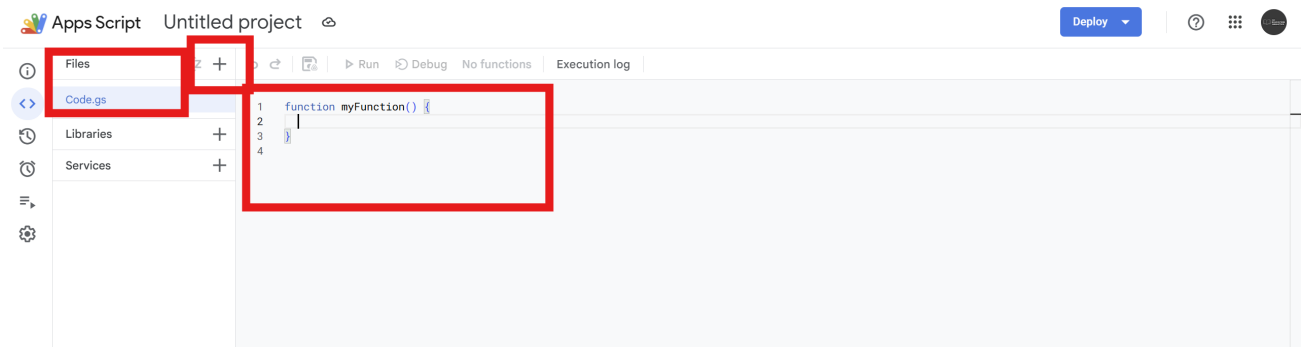
# Setup Screenshot Reference

Visual cues for the first web app setup and deployment.

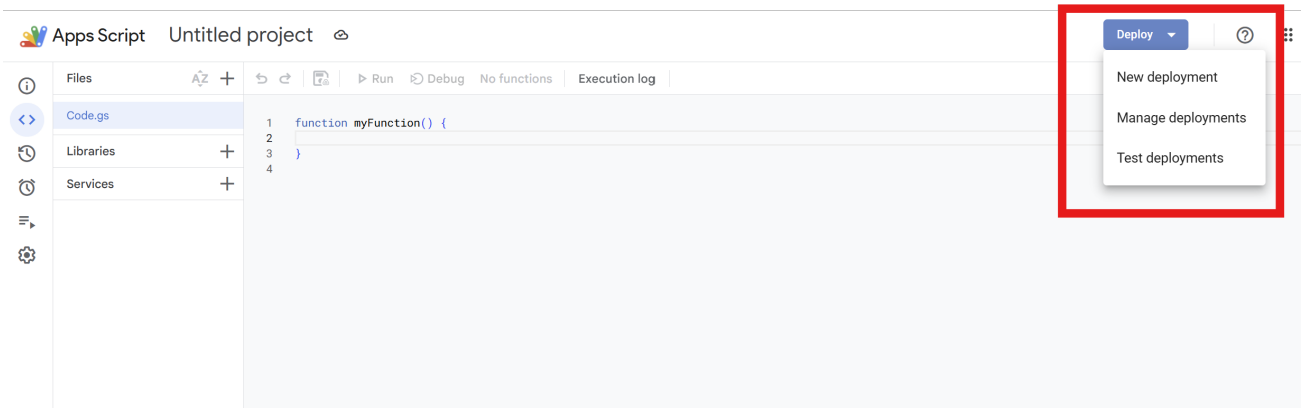
## Setup Screenshots




1. **Open Apps Script from the Google Sheet by choosing Extensions > Apps Script.**



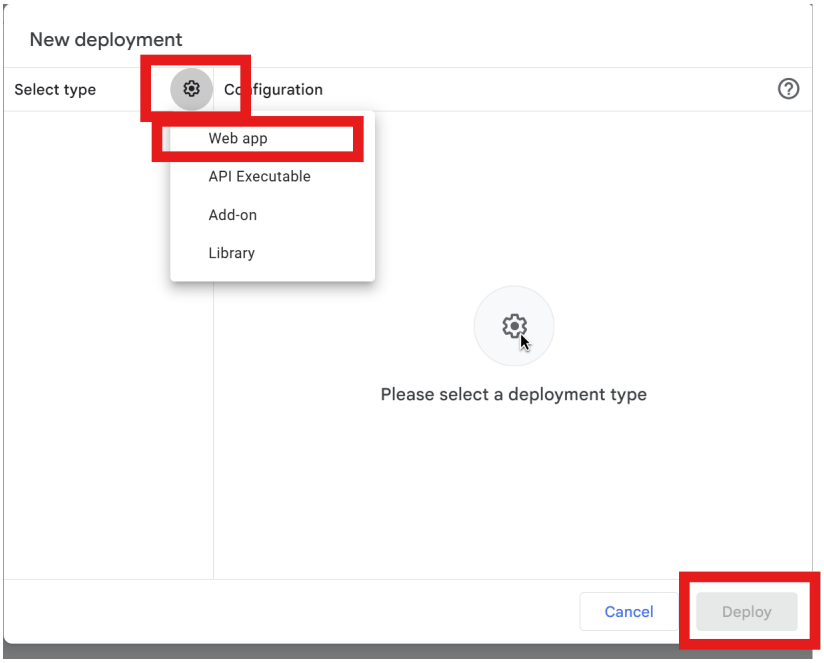

2. **Keep the project organized with server code in Code.gs and page code in index.html.**



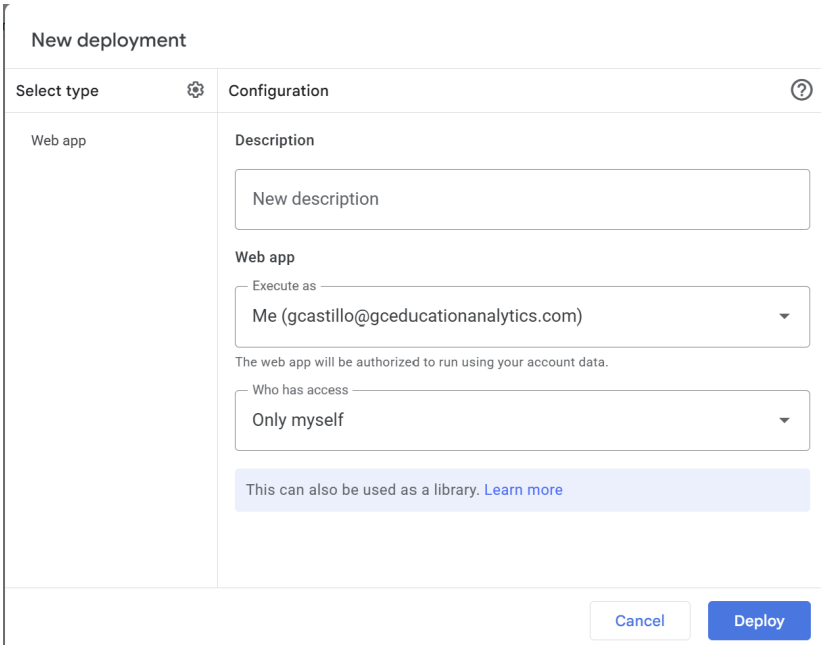

3. **Use the Deploy menu when you are ready to create or update the web app deployment.**

# Setup Screenshot Reference

Deployment screens beginners should recognize.




**4. Choose Web app as the deployment type when creating a new deployment.**




**5. Review the web app settings carefully, especially Execute as and Who has access.**